Computer Network Course

Bahareh Einollahi

Rayhaneh Einollahi

Tir 1405

Mini_TCP implementation using UDP/IP

0-Basic Design and Code Structure

Control Flags

This section sets up the core rules of our protocol. `MAX_PAYLOAD_SIZE` restricts the actual message data to a maximum of 1024 bytes. `HEADER_SIZE` is exactly 13 bytes, matching the size of the header fields added together. The flags use specific bits (hex values `0x01`, `0x02`, `0x04`) to signal what kind of packet this is—whether it is carrying normal data, acknowledging a packet, or closing a connection.

```
constexpr uint16_t MAX_PAYLOAD_SIZE = 1024;
constexpr size_t HEADER_SIZE = 13; // 4 + 4 + 2 + 2 + 1 bytes

constexpr uint8_t FLAG_DATA = 0x01;
constexpr uint8_t FLAG_ACK  = 0x02;
constexpr uint8_t FLAG_FIN  = 0x04;
```

packet Header & packet

These two structs define how our data is organized. `PacketHeader` groups the metadata needed to manage the connection, including tracking numbers (sequence and acknowledgment), data length, an error-checking checksum, and control flags. The `Packet` struct combines this 13-byte header with a fixed-size `payload` buffer where the actual data lives.

```
struct PacketHeader {
    uint32_t seq_num;
    uint32_t ack_num;
    uint16_t length;
    uint16_t checksum;
    uint8_t flags;
};
struct Packet {
    PacketHeader header;
    char payload[MAX_PAYLOAD_SIZE];
};
```

compute_checksum

This function calculates a standard 16-bit Internet Checksum to detect data corruption during transit. It loops through the data 2 bytes at a time, combining them into 16-bit words and adding them to a running sum. If the total number of bytes is odd, it handles the last byte safely. It then squashes any overflow bits back into the lower 16 bits and uses the bitwise NOT operator (~) to flip the bits for the final answer.

```
inline uint16_t compute_checksum(const uint8_t* data, size_t length) {
    uint32_t sum = 0;
    for (size_t i = 0; i < length - 1; i += 2) {
        sum += (data[i] << 8) | data[i + 1];
    }
    if (length & 1) {
        sum += (data[length - 1] << 8);
    }
    while (sum >> 16) {
        sum = (sum & 0xFFFF) + (sum >> 16);
    }
    return ~static_cast<uint16_t>(sum);
}
```

Packet Serialization

Serialization takes our clean C++ `Packet` struct and turns it into a flat array of bytes that can be sent over a network socket. Because different computers read numbers in different byte orders, functions like `htonl` and `htons` convert our values into Network Byte Order (Big Endian). We then use `std::memcpy` to safely copy each header field and the payload into their correct, exact positions in a byte vector.

```
inline std::vector<uint8_t> serialize_packet(const Packet& packet) {
    std::vector<uint8_t> buffer(HEADER_SIZE + packet.header.length);

    uint32_t net_seq = htonl(packet.header.seq_num);
    uint32_t net_ack = htonl(packet.header.ack_num);
    uint16_t net_len = htons(packet.header.length);
    uint16_t net_chk = htons(packet.header.checksum);

    std::memcpy(&buffer[0], &net_seq, 4);
    std::memcpy(&buffer[4], &net_ack, 4);
    std::memcpy(&buffer[8], &net_len, 2);
    std::memcpy(&buffer[10], &net_chk, 2);
    buffer[12] = packet.header.flags;

    if (packet.header.length > 0) {
        std::memcpy(&buffer[HEADER_SIZE], packet.payload, packet.header.length);
    }

    return buffer;
}
```

Header fields and their usage:

Field / Flag Name	Data Type / Value	Size	Functional Purpose
seq_num	uint32_t	4 Bytes	Sequence Number: Tracks the position of the data in the byte stream. It helps the receiver put packets back in the right order and spot missing data.
ack_num	uint32_t	4 Bytes	Acknowledgment Number: Tells the other side the sequence number of the next packet we are expecting to receive, confirming everything before it arrived safely.
length	uint16_t	2 Bytes	Payload Length: Stores the exact size of the incoming data block (anywhere from 0 to 1024 bytes) so the program knows exactly where the data ends.
checksum	uint16_t	2 Bytes	Error Check: Holds a mathematical value calculated from the whole packet. The receiver recalculates this to verify the data wasn't corrupted during transmission.
flags	uint8_t	1 Byte	Control Flags: A byte where individual bits are turned on or off to change the packet's type and update the connection state (defined below).
*FLAG_DATA	0x01	*(Bit 0)*	Indicates that this packet is carrying a normal data payload.
* FLAG_ACK	0x02	*(Bit 1)*	Indicates that this packet is an acknowledgment message confirming received data.
* FLAG_FIN	0x04	*(Bit 2)*	Indicates a finish request, used to cleanly close down the network connection.

1.Stop & wait trustable transfer

How Stop & wait works?

Stop-and-Wait is a simple transmission protocol in which the sender transmits one packet and then waits until it receives an acknowledgment (ACK) from the receiver before sending the next packet. If the ACK arrives successfully, the sender sends the next packet. If the ACK is not received within a certain timeout period, the sender assumes that the packet or its ACK was lost and retransmits the same packet. Since only one packet can be in transit at a time, the protocol is simple and reliable, but it does not use the network efficiently, especially when the communication delay is high.

Implementation

With stop and wait every packet will be sent and then waits for the ack to arrive.

We implemented this in the sender so that it works as expected. The time out is set to a custom value, so if the single sent packet doesn't send the ack in this time interval the sender is going to send (retransmit) the packet again. Below is the results after transmission:

```
[17:30:24.322] [ACK ] Received valid ACK for DATA (Seq: 46080)
[17:30:24.322] [SEND] Sent DATA Packet (Seq: 47104, Bytes: 1037)
[17:30:24.323] [ACK ] Received valid ACK for DATA (Seq: 47104)
[17:30:24.323] [SEND] Sent DATA Packet (Seq: 48128, Bytes: 185)
[17:30:24.323] [ACK ] Received valid ACK for DATA (Seq: 48128)
[17:30:24.323] [SEND] Sent FIN Packet (Seq: 48300, Bytes: 13)
[17:30:24.323] [ACK ] Received valid ACK for FIN (Seq: 48300)

======================================
       PHASE 1 TRANSFER METRICS
======================================
Total Packets Sent (incl. retrans): 49
Number of Retransmissions         : 0
Total File Transfer Time          : 23.8222 ms
======================================
```

The file was divided to 49 packets and because the link has no problem no retransmissions are needed the total File transfer time also show how much the whole transfer took.

To compare the send and received files we used the diff command and as expected no difference was there:

```
/ca4$ diff input.txt output.txt
/ca4$
```

We also switched to more oop code so it would be easier to change for the next phases

2. Sliding Window Implementation

How sliding window works?

Sliding window is a transmission method that allows the sender to send multiple packets before receiving acknowledgments (ACKs) for each one. The sender can have a fixed number of unacknowledged packets in the network at the same time, called the window size. As ACKs are received, the window moves forward, allowing new packets to be sent while keeping the number of outstanding packets within the window limit. If a packet is lost, it is retransmitted, and the window continues to advance after the missing packet is successfully acknowledged. By allowing several packets to be transmitted at once instead of waiting for an ACK after every packet, the sliding window protocol makes better use of the network and improves throughput.

Implementation

In this phase, we modified our implementation to support a sliding window instead of sending only one packet at a time. This required keeping track of multiple packets that had been sent but not yet acknowledged. We also updated the timeout handling so that packets that are still waiting for an ACK are retransmitted when necessary.

```
    bool run_sliding_window(std::ifstream& infile, uint32_t& seq_num) {
        const size_t WINDOW_SIZE = 5;

        struct WindowPacket {
            uint32_t seq_num;
            uint32_t expected_ack;
            std::vector<uint8_t> data;
        };
        std::vector<WindowPacket> window;
```

we also changed the receiver so that the out of order packets are buffered not dropped to implement the cumulative way:

```
      void handle_data_packet(const Packet& pkt, const sockaddr_in& src_addr, socklen_t addr_len) {
        if (pkt.header.seq_num == expected_seq_num) {
            outfile.write(pkt.payload, pkt.header.length);
            expected_seq_num += pkt.header.length;
            log_msg("RCVD", "Accepted Seq: " + std::to_string(pkt.header.seq_num));

            while (packet_buffer.count(expected_seq_num)) {
                Packet next_pkt = packet_buffer[expected_seq_num];
                outfile.write(next_pkt.payload, next_pkt.header.length);
                packet_buffer.erase(expected_seq_num);
                expected_seq_num += next_pkt.header.length;
            }
        } else if (pkt.header.seq_num < expected_seq_num) {
            log_msg("WARN", "Received duplicate Seq: " + std::to_string(pkt.header.seq_num) + ". Dropping
data payload.");
        } else {
            packet_buffer[pkt.header.seq_num] = pkt;
            log_msg("WARN", "Out-of-order Seq: " + std::to_string(pkt.header.seq_num) + " buffered.");
        }
```

As you can see the first 5 packets are sent together, then the acks arrive and another packet can be sent:

```
[18:00:32.525] [INFO] Starting Sliding Window transmission...
[18:00:32.525] [SEND] Sent DATA Packet (Seq: 0, Bytes: 1037)
[18:00:32.525] [SEND] Sent DATA Packet (Seq: 1024, Bytes: 1037)
[18:00:32.525] [SEND] Sent DATA Packet (Seq: 2048, Bytes: 1037)
[18:00:32.525] [SEND] Sent DATA Packet (Seq: 3072, Bytes: 1037)
[18:00:32.525] [SEND] Sent DATA Packet (Seq: 4096, Bytes: 1037)
[18:00:32.526] [ACK ] Window advanced to ACK: 1024
```

Until the end that acks of the last 5 packets arrive one after another:

```
[18:00:32.540] [ACK ] Window advanced to ACK: 41984
[18:00:32.540] [SEND] Sent DATA Packet (Seq: 46080, Bytes: 1037)
[18:00:32.541] [ACK ] Window advanced to ACK: 43008
[18:00:32.541] [SEND] Sent DATA Packet (Seq: 47104, Bytes: 1037)
[18:00:32.541] [ACK ] Window advanced to ACK: 44032
[18:00:32.541] [SEND] Sent DATA Packet (Seq: 48128, Bytes: 185)
[18:00:32.541] [ACK ] Window advanced to ACK: 45056
[18:00:32.542] [ACK ] Window advanced to ACK: 46080
[18:00:32.542] [ACK ] Window advanced to ACK: 47104
[18:00:32.542] [ACK ] Window advanced to ACK: 48128
[18:00:32.542] [ACK ] Window advanced to ACK: 48300
[18:00:32.542] [SEND] Sent FIN Packet (Seq: 48300, Bytes: 13)
[18:00:32.542] [ACK ] Received valid ACK for FIN (Seq: 48300)
```

The results show that this algorithm takes less time to transmit all packets because at any time there are 5 packets in flight instead of one:

```
=======================================
        PHASE 1 TRANSFER METRICS
=======================================
Total Packets Sent (incl. retrans): 49
Number of Retransmissions         : 0
Total File Transfer Time          : 20.9231 ms
=======================================
```

Thoughput is also calculated using this formula:

$$\text{Throughput} = \frac{\text{Total File Size (KB)}}{\text{Total Transfer Time (s)}}$$

So the throughput would be 2.39 MB/s

Question: why Sliding Window usually performs better than Stop-and-Wait?

Sliding Window usually performs better than Stop-and-Wait because the sender does not have to wait for an acknowledgment after every packet. Instead, it can keep sending packets until the transmission window is full. This keeps the network busy and makes better use of the available bandwidth, especially when the delay between the sender and receiver is high. As a result, less time is wasted waiting for ACKs, file transfers finish faster, and the overall throughput is higher than with the Stop-and-Wait protocol.

3. Introduce Loss and Delay

The receiver was modified to parse --loss and --delay flags. A pseudo-random number generator drops valid incoming packets based on the loss probability, and introduces a fixed delay right before sending ACKs.

The outputs of each tested condition are represented in this table:

Test Case	Loss Rate	Added Delay	Transfer Time	Retransmissions	Throughput
Baseline	0%	0 ms	*21.34 ms*	0	*2.39 MB/s*
Effect of Loss	10%	0 ms	*3 s*	15	*21 KB/s*
Effect of Delay	0%	100 ms	*4.9s*	0	*10 KB/s*
Combined	10%	100 ms	*10.5 s*	30	*7 KB/s*

The 10% loss rate hurts speed bad (from 2.39 MB/s to 21 KB/s) because when packets drop, sender stops and waits for timeout a long time. Also, the 100 ms delay makes the Round-Trip Time (RTT) too long, so sender just sits there waiting for ACKs before it can send more data. When you put loss and delay together, it is a disaster: the delay makes everything slow, and because it is slow, more packets get lost again, doubling retransmissions to 30 and making speed drop to a terrible 7 KB/s.

QUESTION: why increasing loss increases retransmissions and decreases throughput

When the receiver or network drops a packet, the sender's timeline breaks and triggers a timeout while waiting for the missing ACK. Assuming the data was lost, the sender is forced to retransmit the exact same packet, and higher loss rates exponentially increase the likelihood of both initial data and duplicate ACKs being dropped, causing a cascading spike in retransmissions. This process destroys throughput because retransmitted packets consume the network link's capacity to resend old data instead of moving the file transfer forward. Furthermore, every dropped packet introduces idle dead-time, forcing the sender to stop transmitting new data and sit completely inactive during the timeout interval, which heavily inflates the total transfer time.

4. Congestion Avoidance & Slow start

The concepts of ssthresh & cwnd

cwnd (Congestion Window) is a variable maintained by the sender that determines how many packets can be sent into the network without waiting for acknowledgments (ACKs). It changes dynamically based on the current network conditions. A larger cwnd allows more packets to be in transit at the same time, increasing throughput, while a smaller cwnd reduces the sending rate to avoid overloading the network.

ssthresh (Slow Start Threshold) is a value that determines when the sender should switch from **Slow Start** to **Congestion Avoidance**. While cwnd is smaller than ssthresh, the sender increases cwnd rapidly using Slow Start. Once cwnd reaches or exceeds ssthresh, the sender switches to Congestion Avoidance, where cwnd grows more slowly. If congestion is detected, such as after a timeout, ssthresh is reduced (typically to half of the current cwnd), and cwnd is reset to one packet before Slow Start begins again.

Implementation

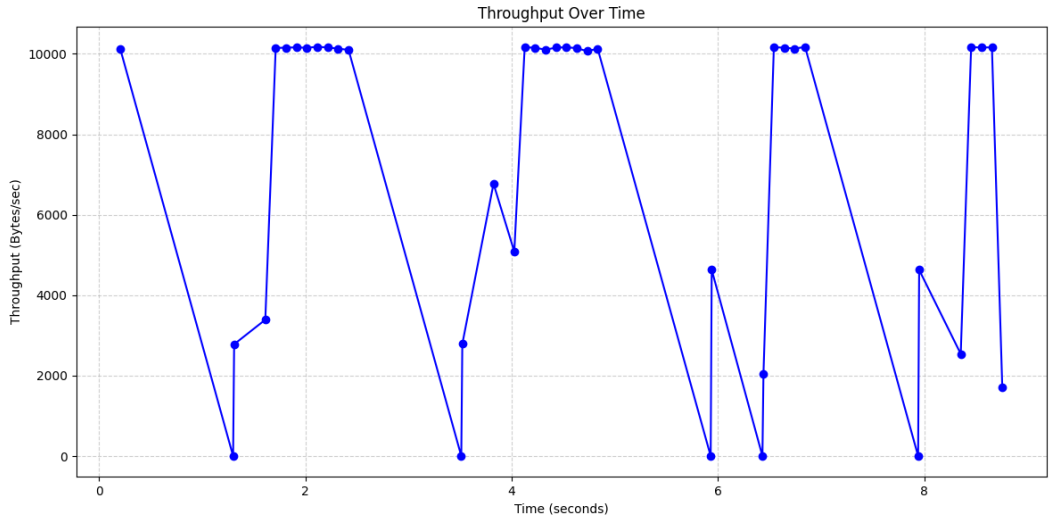
Congestion control was added to the TCP simulation by maintaining a congestion window (cwnd) that limits the number of packets the sender can have in flight. At the beginning of a connection or after a timeout, the sender enters **Slow Start**, where cwnd starts at one packet and increases rapidly by one packet for each ACK received, resulting in exponential growth. When cwnd reaches the slow start threshold (ssthresh), the sender switches to **Congestion Avoidance**, where cwnd grows more slowly by approximately one packet per round-trip time, producing linear growth. If packet loss is detected through a timeout, ssthresh is reduced, cwnd is reset to one packet, and the sender returns to Slow Start before gradually increasing the transmission rate again.

Results

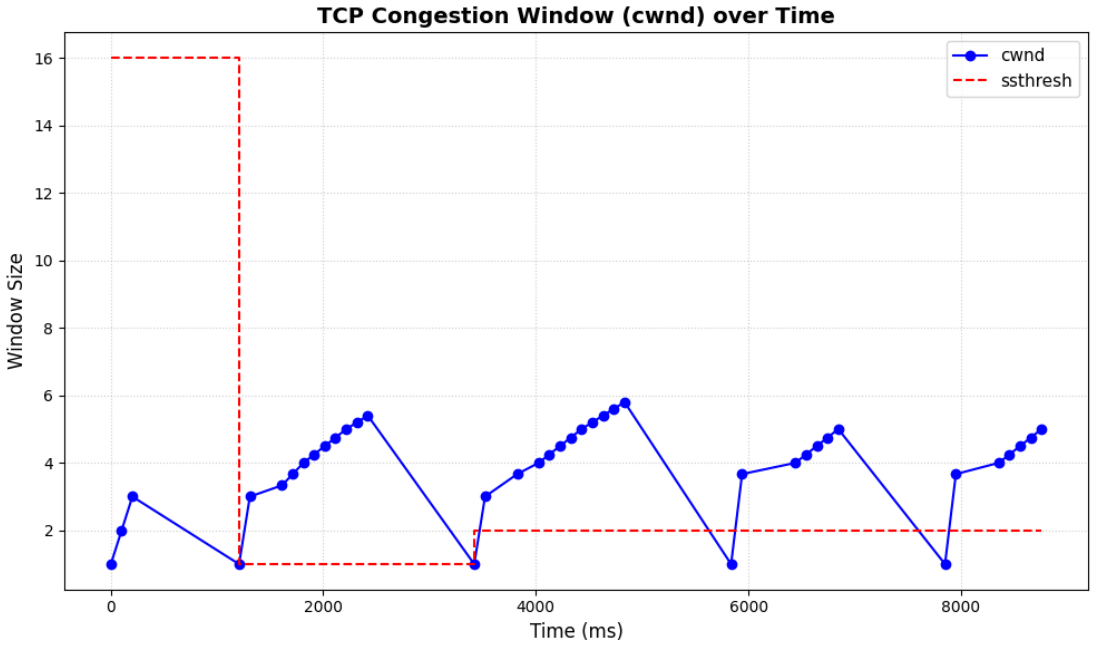
The cwnd.csv and log of events are generated when the code runs and is currently present in the project zip

In order to draw the throughput over time graph we first wrote a python code to extract the data from the log and figures out the transfer speed over time. First, it looks only for the lines marked "ACK," which mean the other side successfully received some data. It converts the log's timestamps from milliseconds into normal seconds, and then checks how many new bytes were confirmed since the last "ACK" line. By dividing those new bytes by the time that passed between the two messages, it calculates the network speed in bytes per second. If it notices a long pause caused by a timeout error, it smartly adds a zero-speed marker to show the connection dropped.

Finally, it feeds all these calculated times and speeds into a graphing tool called matplotlib, which connects the dots to draw a line chart showing your network's speed ups and downs.:



In order to draw the cwnd over time we are going to use the csv file we generated during the transfer and a custom python code to generate the graph:



QUESTION: why TCP reduces the cwnd value during a timeout and why it starts again from Slow Start?

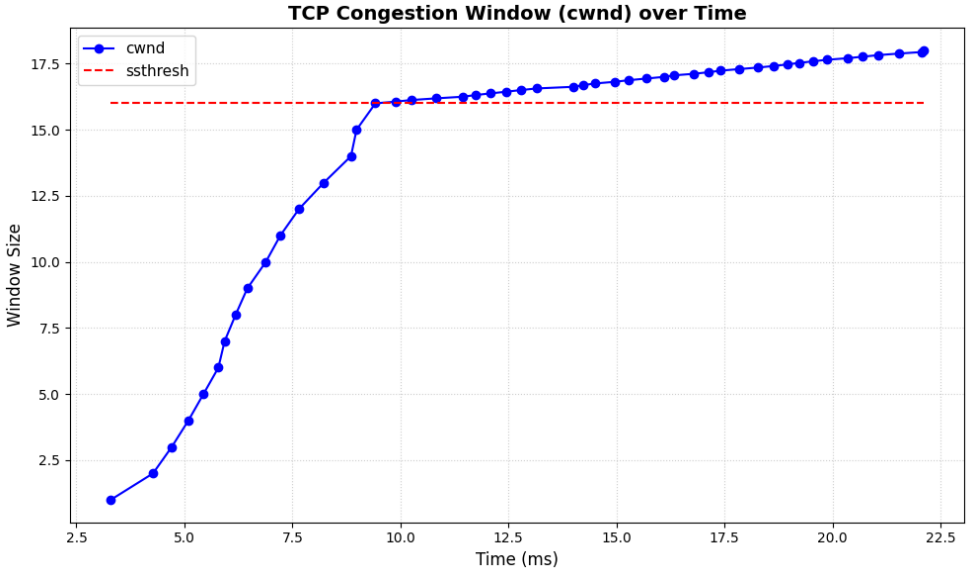
When a timeout happens, TCP realizes that packets are getting completely lost because the network router's buffer is totally full and overflowing—kind of like a massive traffic jam. To stop making the traffic jam worse, TCP immediately crushes its cwnd (sending speed) all the way back down to 1 so it stops flooding the network. It then resets into **Slow Start** to safely test the waters again. Since it has no idea how bad the traffic jam still is, starting from 1 and rapidly doubling the speed allows it to quickly find a safe, stable speed without accidentally overloading the network all over again.

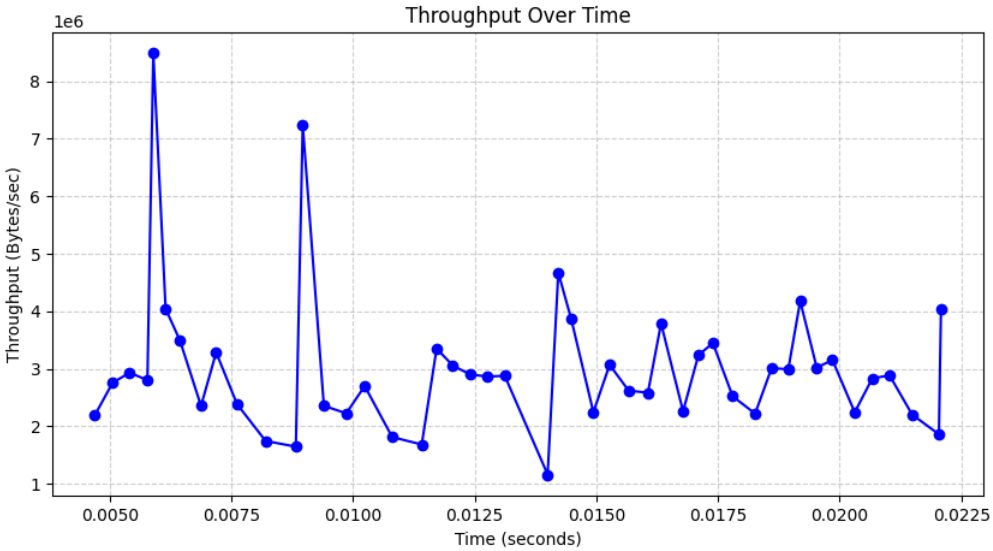
5. Performance analysis and charting

Case1: Loss = 0%, Delay = 0 ms

In this case the transfer is going to face zero timeouts and no retransmissions are required:

Metric	Value
Total Packets Sent (incl. retrans)	49
Number of Retransmissions	0
Total File Transfer Time	23.8173 ms
Throughput	2106.70 KB/s


TCP Congestion Window (cwnd) over Time
cwnd
ssthresh
Window Size
Time (ms)
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
22.5

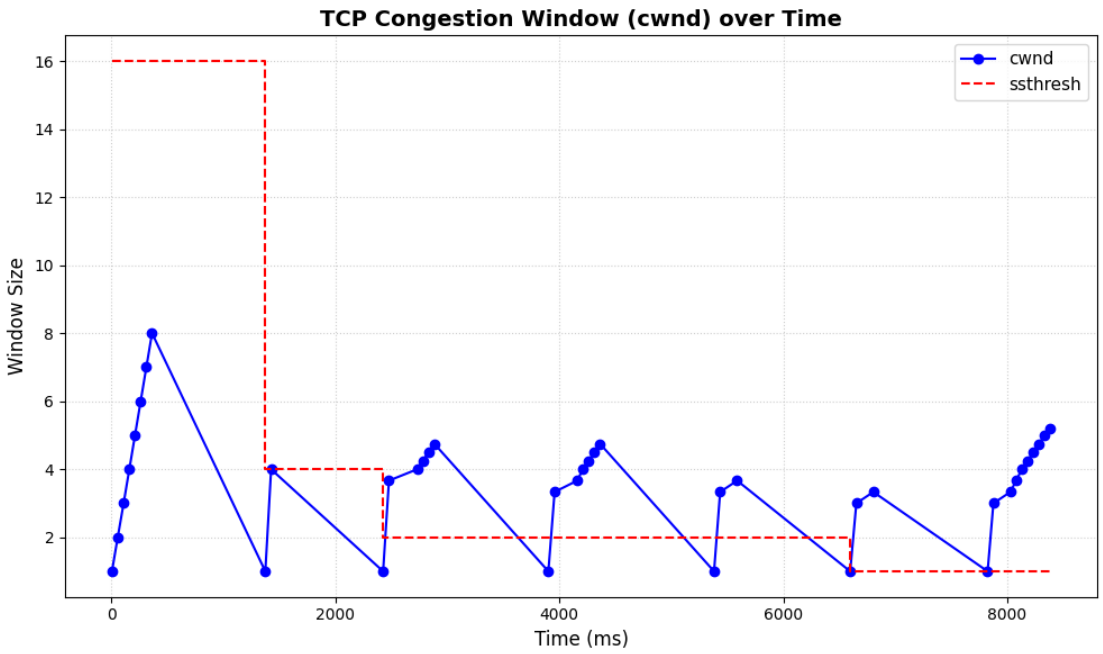

Throughput Over Time
1e6
Throughput (Bytes/sec)
Time (seconds)
1
2
3
4
5
6
7
8
0.0050
0.0075
0.0100
0.0125
0.0150
0.0175
0.0200
0.0225

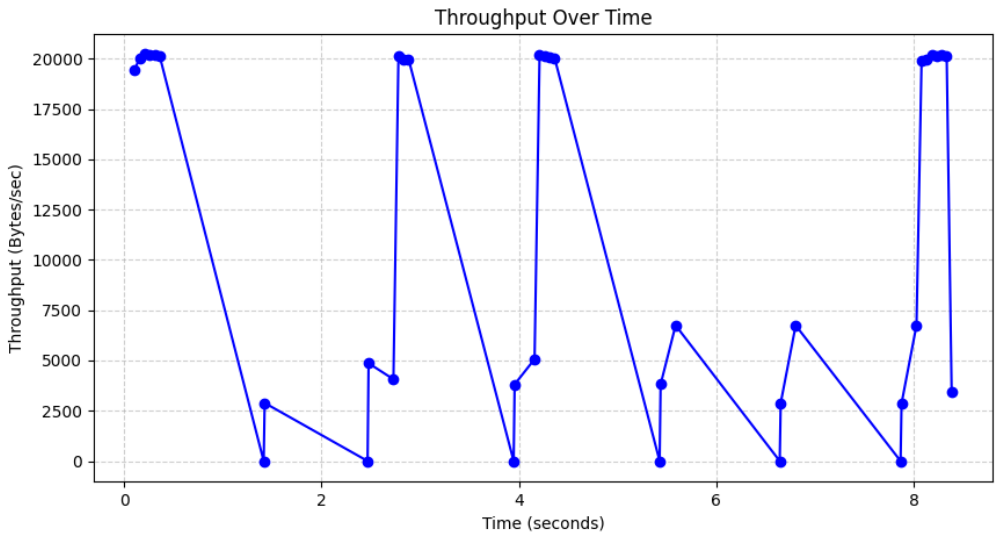
Case2: Loss = **10**%, Delay = **50** ms

In this case the transfer is going to face 0.1 timeouts and retransmission are going to happen :

The total transfer time is going to be more because of the delay and lost packets that need retransmission.

Metric	Value
Total Packets Sent (incl. retrans)	76
Number of Retransmissions	27
Total File Transfer Time	8437.04 ms
Throughput	9.22 KB/s



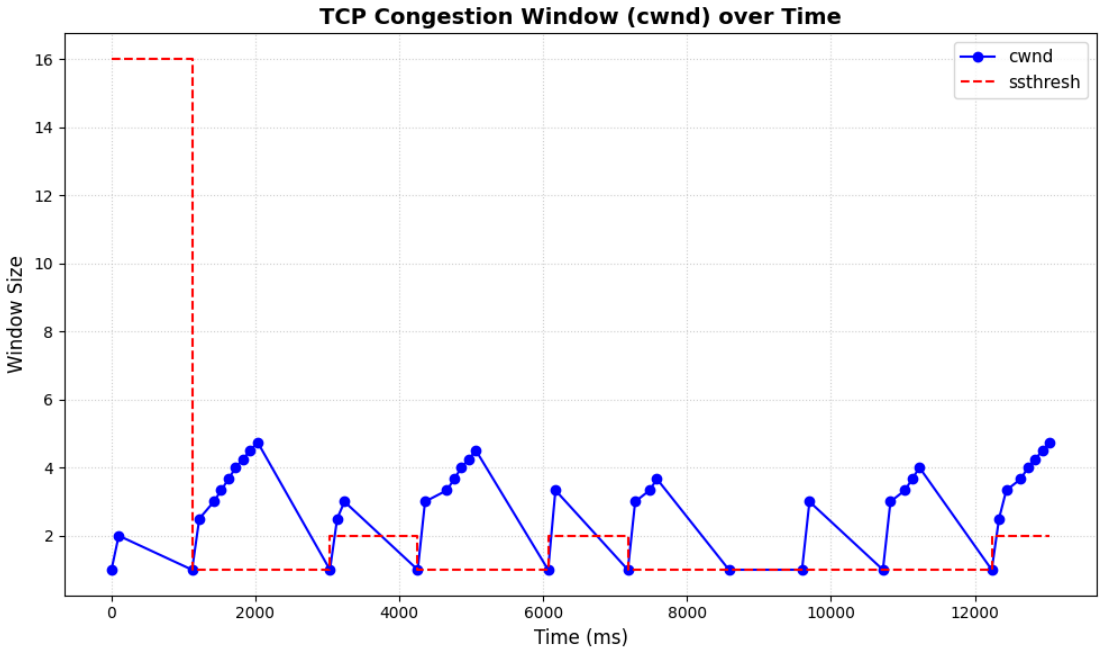


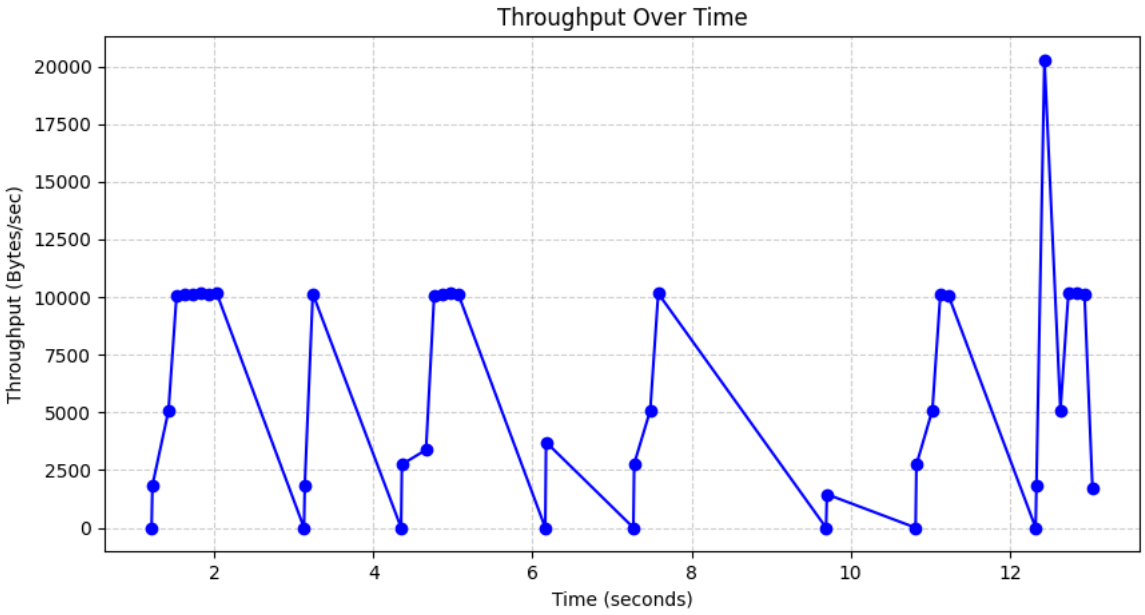
Case3: Loss = 20%, Delay = 100 ms

In this case the transfer is going to face many more timeouts and more retransmissions are required so the delay is going to be more than the previous case and the throughput would be less!

The file still is transferred successfully!

Metric	Value
Total Packets Sent (incl. retrans)	78
Number of Retransmissions	29
Total File Transfer Time	13146 ms
Throughput	6.08 KB/s


TCP Congestion Window (cwnd) over Time
cwnd
ssthresh
Window Size
Time (ms)
0
2000
4000
6000
8000
10000
12000
2
4
6
8
10
12
14
16


Throughput Over Time
Throughput (Bytes/sec)
Time (seconds)
0
2500
5000
7500
10000
12500
15000
17500
20000
2
4
6
8
10
12

6. Fast Retransmission and Fast Recovery

Fast Retransmit and Fast Recovery are optimization techniques designed to handle packet loss efficiently without slowing down data transmission. Normally, a sender waits for a timeout timer to expire before resending a lost packet, which causes a noticeable delay. With **Fast Retransmit**, if the sender receives three identical duplicate ACKs from the receiver, it realizes a specific packet was dropped along the way and immediately resends it without waiting for the timer.

After this, **Fast Recovery** keeps the transmission moving smoothly; instead of resetting the congestion window all the way back to one and restarting slowly, the sender reduces the window size to half of its previous value and continues sending new data from this higher baseline, allowing the network to maintain high throughput and recover much faster.

Outputs:

- Fast retransmission happening:

```
[12:47:34.620] [ACK] recevied ACK seq number: 7168
[12:47:34.721] [ACK] recevied ACK seq number: 7168
[12:47:34.821] [ACK] recevied ACK seq number: 7168
[12:47:34.821] [FAST_RETRANSMIT] 3 Duplicate ACKs received for 7168
[12:47:34.821] [RETRANSMIT] Fast Retransmitted DATA Packet (Seq: 7168)
```

- Comparing **total transmission time** and **number of timeouts** for fast retransmission and normal mode:

  We can see number of timeouts and total time has reduced when added fast retransmission.

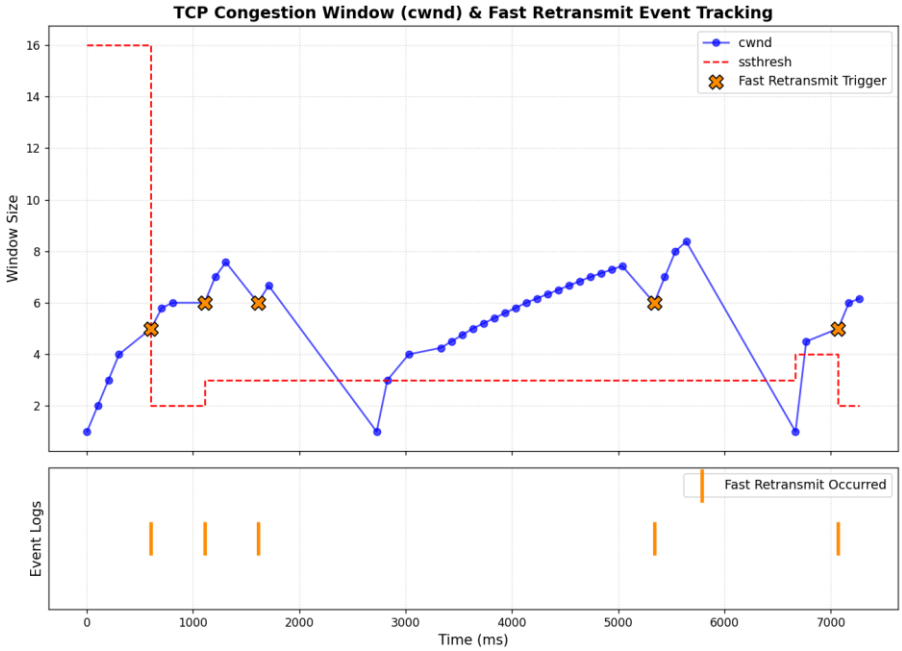
  Fast retransmission:

```
========================================
        TRANSFER METRICS
========================================
Total Packets Sent (incl. retrans): 54
Total Timeouts Triggered          : 1
Fast Retransmits Triggered        : 3
Number of Retransmissions         : 5
Total File Transfer Time          : 5932.96 ms
Throughput                        : 9.32 KB/s
========================================
```

  Normal congestion control:

```
========================================
        TRANSFER METRICS
========================================
Total Packets Sent (incl. retrans): 74
Total Timeouts Triggered          : 2
Fast Retransmits Triggered        : 0
Number of Retransmissions         : 25
Total File Transfer Time          : 6627.53 ms
Throughput                        : 11.43 KB/s
========================================
```

- *Cwnd chart and showing fast retransmission points:*

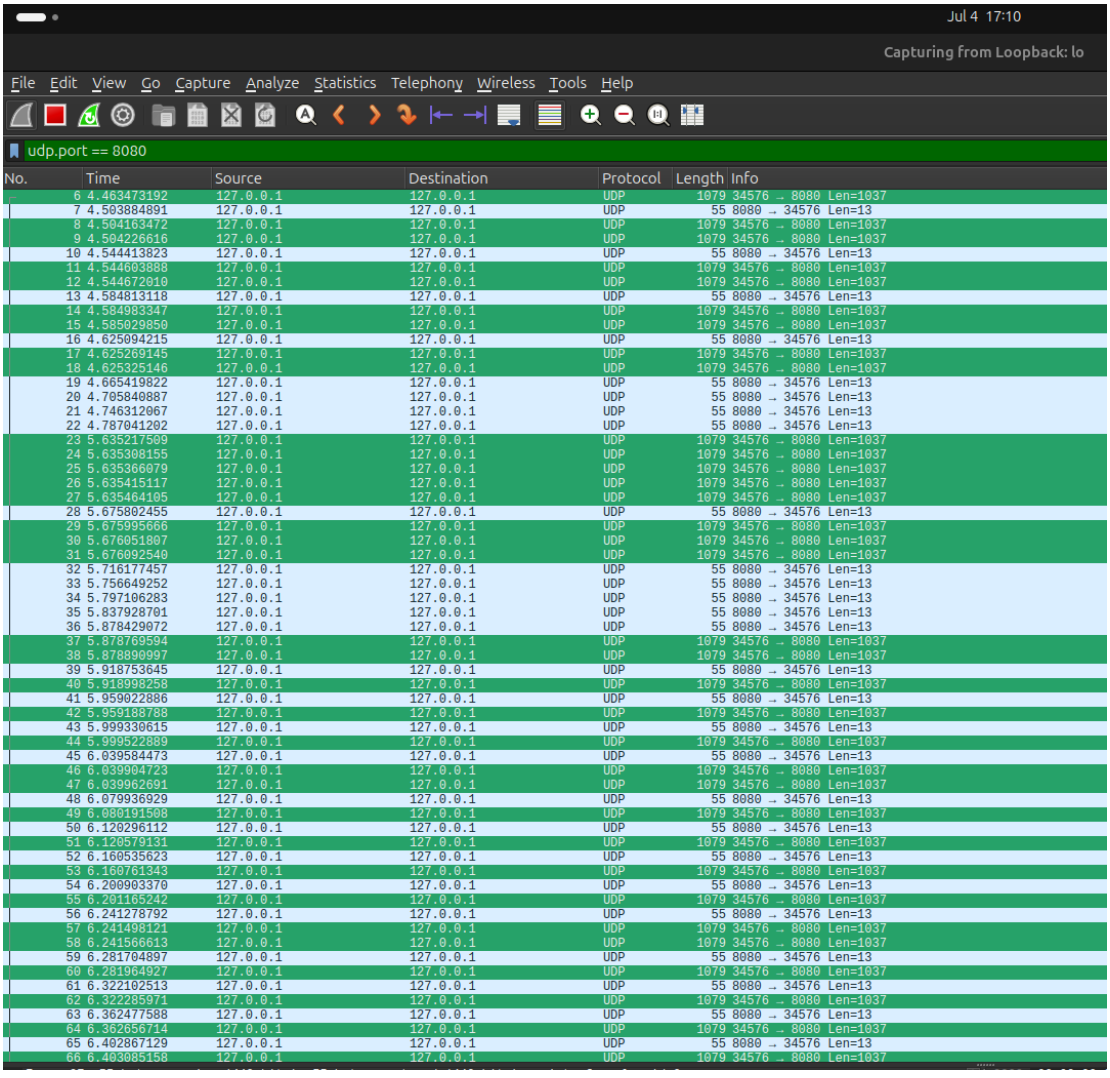


7.BONUS

Wireshark

In order to capture the traffic sent and received, because both are on a single network and use the loopback address (127.0.0.1) we are going to capture the lo interface which is going to show us the traffic we are looking for. In order to filter to just the traffic in and out from port 8080 reciever, we used the filter udp.port == 8080.

Then the inbound and outbound traffic wasn't distinguishable so we colored the send packets in green and the received packets(in sender) with blue. Below is the result for the phase4 congestion control that show how the cwnd gets exponentially larger and then gets back to one once a timer hits timeout.



Several connections simultaneously

Because the receiver is set to the specific port via command line and the sender has no limitations in which port to use. We can achieve this by running several instances of receiver and sender on each machine just using different ports.

As we can see a computer can use one sender process to send a file to another computer with port a, and another sender for sending a different file which sends to port b. so the other computer would be able to receive both files simultaneously.

In the results below the two left processes are receiving and the two on the right are sending. The result of the diff shows that the two files are the same and have no difference:

./receiver.o 8080 output1.txt --loss 0.1 --delay 100

./receiver.o 8081 output2.txt --loss 0.1 --delay 100

./sender.o 127.0.0.1 8080 input1.txt sliding-window

./sender.o 127.0.0.1 8081 input2.txt sliding-window

